Resumen

El diagrama de clase describe los tipos de objetos que hay en el sistema y las diversas clases de relaciones estáticas que existen entre ellos. Hay dos tipos principales de relaciones estáticas:

- **Asociaciones** (por ejemplo, un cliente puede rentar diversas videocintas).
- **Subtipos** (una enfermera es un tipo de persona).

Los diagramas de clase también muestran los atributos y operaciones de una clase y las restricciones a que se ven sujetos, según la forma en que se conecten los objetos.

Arriba 

Índice de Contenidos

- Diagramas de clase: fundamentos
  -
    - Resumen
    - Mapa General
    - Índice de Contenidos
    - Introducción
    - Perspectivas
    - Asociaciones
    - Atributos
    - Operaciones
    - Generalización
    - Reglas de restricción
    - Cuándo emplear los diagramas de clases
    - Para mayor información
    - Índice
    - Referencia Bibliográfica
    - Pie del Documento

Introducción

La técnica del **diagrama de clase** se ha vuelto medular en los métodos orientados a objetos. Virtualmente, todos los métodos han incluido alguna variación de esta técnica.

El diagrama de clase, además de ser de uso extendido, también está sujeto a la más amplia gama de conceptos de modelado. Aunque los elementos básicos son necesarios para todos, los conceptos avanzados se usan con mucha menor frecuencia. Por eso, he dividido mi estudio de los diagramas de clase en dos partes; los fundamentos (en el presente capítulo) y los conceptos avanzados los conceptos avanzados (véase el capítulo siguiente).

El diagrama de clase describe los tipos de objetos que hay en el sistema y las diversas clases de relaciones estáticas que existen entre ellos. Hay dos tipos principales de relaciones estáticas:

- **Asociaciones** (por ejemplo, un cliente puede alquilar diversas videocintas).
- **Subtipos** (una enfermera es un tipo de persona).

Los diagramas de clase también muestran los atributos y operaciones de una clase y las restricciones a que se ven sujetos, según la forma en que se conecten los objetos.

Los diversos métodos OO utilizan terminologías diferentes (y con frecuencia antagónicas) para estos conceptos. Se trata de algo sumamente frustrante pero inevitable, dado que los lenguajes OO son tan desconsiderados como los métodos. Es en esta área que el UML aportará algunos de sus mayores beneficios, al simplificar estos diferentes diagramas.

La figura 4-1 muestra un diagrama de clase típico.

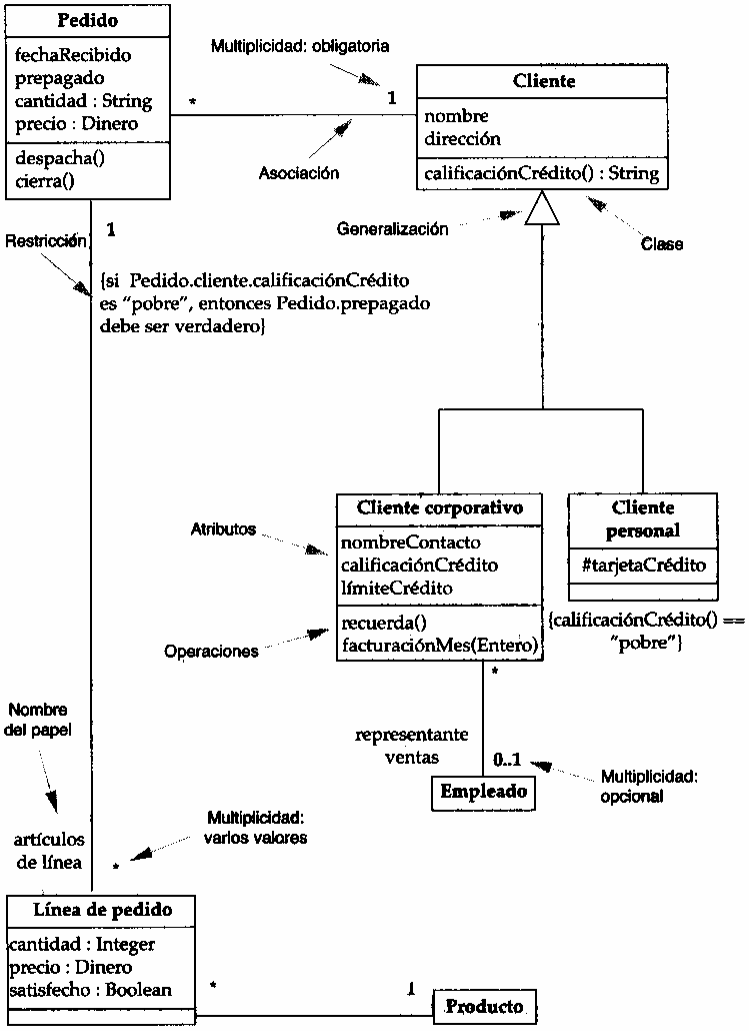


Figura 4-1: *Diagrama de clase*

Figura 4-1 Diagrama de clase

Arriba

Perspectivas

Antes de empezar a describir los diagramas de clase, quisiera señalar una importante sutileza sobre el modo en que se usan. Tal sutileza generalmente no está documentada, pero tiene sus repercusiones en el modo en que debe interpretarse un diagrama, ya que se refiere a lo que se va a describir con un modelo.

Siguiendo la recomendación de Steve Cook y John Daniels (1994), considero que hay tres perspectivas que se pueden manejar al dibujar diagramas de clase (o, de hecho, cualquier modelo, aunque esta división se advierte de modo especial en relación con los diagramas de clase).

- **Conceptual.** Si se adopta la perspectiva conceptual, se dibuja un diagrama que represente los conceptos del dominio que se está estudiando. Estos conceptos se relacionan de manera natural con las clases que los implementan, pero con frecuencia no hay una correlación directa. De hecho, los modelos conceptuales se deben dibujar sin importar (o casi) el software con que se implementarán, por lo cual se pueden considerar como independientes del lenguaje. (Cook y Daniels llaman perspectiva esencial a esto; por mi parte, empleo el término" conceptual", pues se ha usado durante mucho tiempo.)
- **Especificación.** Ahora estamos viendo el software, pero lo que observamos son las interfaces del software, no su implementación, por tanto, en realidad vemos los tipos, no las clases. El desarrollo orientado a objetos pone un gran énfasis en la diferencia entre interfaz e implementación, pero esto con frecuencia se pasa por alto en la práctica, ya que el concepto de clase en un lenguaje OO combina tanto la interfaz como la implementación. Así, a menudo se hace referencia a las interfaces como tipos y a la implementación de esas interfaces como clases. Influidos por este manejo del lenguaje, la mayor parte de los métodos han seguido este camino. Esto está cambiando (Java y CORBA tendrán aquí cierta influencia), pero no con suficiente rapidez. Un tipo representa una interfaz que puede tener muchas implementaciones distintas debido, por ejemplo, al ambiente de implementación, características de desempeño y proveedor. La distinción puede ser muy importante en diversas técnicas de diseño basadas en la delegación.
- **Implementación.** Dentro de esta concepción, realmente tenemos clases y exponemos por completo la implementación. Ésta es, probablemente, la perspectiva más empleada, pero en muchos sentidos es mejor adoptar la perspectiva de especificación.

La comprensión de la perspectiva es crucial tanto para dibujar como para leer los diagramas de clase. Desdichadamente, las divisiones entre las perspectivas no son tajantes y la mayoría de los modeladores no se preocupa por definidas con claridad cuando las dibujan.

Cuando usted dibuje un diagrama, hágalo desde el punto de vista de una sola perspectiva clara. Cuando lea un diagrama, asegúrese de saber desde qué perspectiva se dibujó. Dicho conocimiento es esencial si se quiere interpretar correctamente el diagrama.

La perspectiva no es parte del UML formal, pero considero que es extremadamente valiosa al modelar y revisar los modelos. El UML se puede utilizar con las tres perspectivas. Mediante el etiquetado de las clases con un estereotipo , se puede dar una indicación clara de la perspectiva. Las clases se marcan con «clase de implementación» para mostrar la perspectiva de implementación y con «tipo» para el caso de la perspectiva de especificación y la conceptual. La mayor parte de los usuarios de los métodos OO adopta una perspectiva de implementación, lo cual es de lamentar, porque las otras perspectivas con frecuencia son más útiles.

La tabla 4-1 lista cuatro términos del UML que aparecen en la Figura 4-1 y sus términos correspondientes en otras metodologías bien establecidas.

Tabla 4-1: **Terminología de diagramas de clase**

UML	Clase	Asociación	Generalización	Agregación
Booch	Clase	Usa	Hereda	Contiene
Coad	Clase y objeto	Conexión de instancias	Espec-gen	Parte-todo
Jacobson	Objeto	Asociación por reconocimiento	Hereda	Consiste en
Odell	Tipo de objeto	Relación	Subtipo	Composición
Rumbaugh	Clase	Asociación	Generalización	Agregación
Shlaer/ Mellor	Objeto	Relación	Subtipo	N/A

Arriba

Asociaciones

La figura 4-1 muestra un modelo de clases simple que no sorprenderá a nadie que haya trabajado con un proceso de pedidos. Describiré sus partes y hablaré sobre las maneras de interpretadas, desde las distintas perspectivas.

Comenzaré con las asociaciones. **Las asociaciones** representan relaciones entre instancias de clases (una persona trabaja para una empresa; una empresa tiene cierta cantidad de oficinas).

Desde la **perspectiva conceptual,** las asociaciones representan relaciones conceptuales entre clases. El diagrama indica que un Pedido debe venir de un solo Cliente y que un Cliente puede hacer varios Pedidos en un periodo de tiempo. Cada uno de estos Pedidos tiene varias instancias de Línea de pedido, cada una de las cuales se refiere a un solo Producto.

Cada asociación tiene dos **papeles;** cada papel es una dirección en la asociación. De este modo, la asociación entre Cliente y Pedido contiene dos papeles: uno del Cliente al Pedido; el segundo del Pedido al Cliente.

Se puede nombrar explícitamente un papel mediante una etiqueta. En este caso, el papel en sentido Pedido a Línea de orden se llama Artículos de línea. Si no hay etiqueta, el papel se puede nombrar de acuerdo con la etiqueta de la clase; de esta forma, el papel de Pedido a Cliente se llamará Cliente (en este libro, me refiero a la clase de donde parte el papel como **origen** y a la clase a donde va el papel como **destino.** Esto significa que hay un papel de Cliente cuyo origen es Pedido y cuyo destino es Cliente).

Un papel tiene también una **multiplicidad,** la cual es una indicación de la cantidad de objetos que participarán en la relación dada. En la Figura 4-1, la [*] entre Cliente y Pedido indica que el primero puede tener muchas ordenes asociadas a él; el [1] indica que un Pedido viene de un solo Cliente.

En general, la multiplicidad indica los límites inferior y superior de los objetos participantes. El [*] representa de hecho el intervalo [0.."infinito"]: El Cliente no necesita haber colocado un Pedido, y no hay un tope superior (por lo menos en teoría) para la cantidad de pedidos que puede colocar. El [1] equivale a [1..1]: cada Pedido debe haber sido solicitado por un solo Cliente.

En la práctica, las multiplicidades más comunes son [1], [*], y [0..1] (se puede no tener ninguno o bien tener uno). Para una multiplicidad más general, se puede tener un solo número (por ejemplo, [11] jugadores en un equipo de fútbol), un intervalo (por ejemplo, [2..4] para los participantes de un juego de canasta) o combinaciones discretas de números e intervalos (por ejemplo, [2, 4] para las puertas de un automóvil).

La Figura 4-2 muestra las notaciones de cardinalidad del UML y de los principales métodos pre-UML.

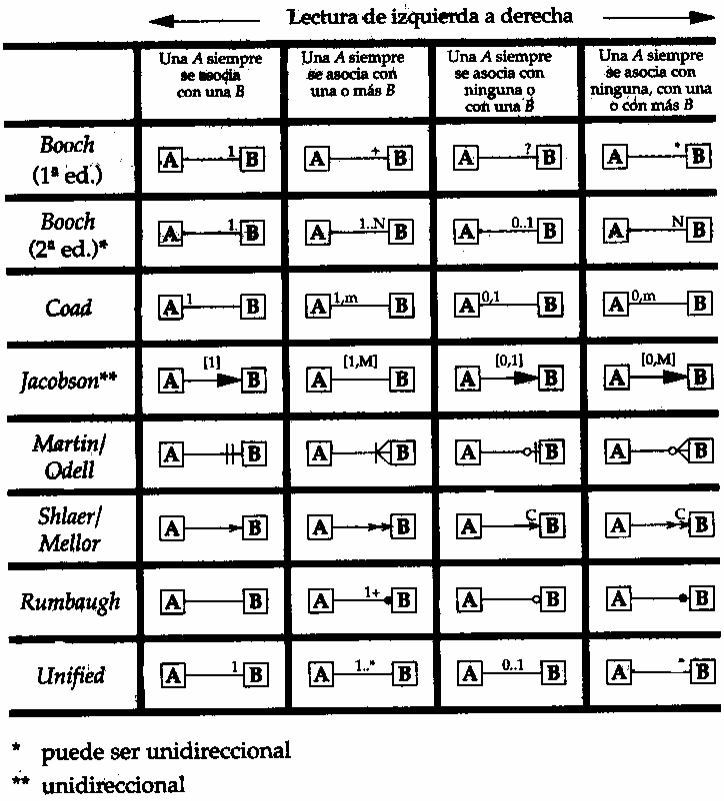


Figura 4-2: *Notaciones de cardinalidad*

Figura 4-2:Notaciones de cardinalidad

Dentro de la **perspectiva de la especificación,** las asociaciones representan responsabilidades.

La Figura 4-1 significa que hay uno o más métodos asociados con Cliente que me proporcionarán los pedidos que ha colocado un Cliente dado. Del mismo modo, existen métodos en Pedido que me permitirán saber qué Cliente colocó tal Pedido y cuales son los Artículos de línea que contiene un Pedido.

Si existen convenciones estándar para nombrar **métodos de consulta,** probablemente sus nombres se puedan deducir del diagrama. Por ejemplo, puedo tener una convención que diga que las **relaciones de un solo valor** se implementan con un método que devuelve el objeto relacionado y que las relaciones de varios valores se implementan mediante una enumeración (iterador) en un **conjunto de objetos relacionados.**

Al trabajar con una convención de nombres como ésta en Java, por ejemplo, puedo deducir la siguiente interfaz para una clase de Pedido:

```
class Pedido {
   public Cliente cliente();
```

```
// Enumeración de líneas de Pedido
public Enumeration lineaspedido() ;
```

Las convenciones de programación, por supuesto, variarán de acuerdo con el lugar y no indicarán todos los métodos, pero le serán a usted de gran utilidad para encontrar el camino.

La Figura 4-1 implica también cierta responsabilidad al poner al día la relación. Debe haber algún modo de relacionar el Pedido con el Cliente. De nuevo, no se muestran los detalles; podría ser que se especifique el Cliente en el constructor del Pedido. O, tal vez, exista un método agregarPedido asociado al Cliente. Esto se puede hacer más explícito añadiendo operaciones al cuadro de clase (tal y como veremos más adelante).

Sin embargo, estas responsabilidades no implican una estructura de datos. A partir de un diagrama a nivel de especificación, no puedo hacer suposición alguna sobre la estructura de datos de las clases. No puedo, ni debo poder decir si la clase Pedido contiene un apuntador a Cliente, o si la clase Pedido cumple su responsabilidad ejecutando algún código de selección que pregunte a cada Cliente si se refiere a un Pedido dado. El diagrama sólo indica la interfaz, y nada más.

Si éste fuera un **modelo de implementación,** ahora daríamos a entender con ello que hay apuntadores en ambos sentidos entre las clases relacionadas. El diagrama diría entonces que Pedido tiene un campo que es un conjunto de apuntadores hacia Línea de pedido y también un apuntador a Cliente. En Java, podríamos deducir algo como lo que exponemos a continuación:

```
class Pedido {
  private Cliente _cliente;
  private Vector _lineaspedido;
class Cliente {
  private Vector _pedidos;
```

En este caso, no podemos inferir nada sobre la interfaz a partir de las asociaciones. Esta información nos la darían las operaciones sobre la clase.

Ahora véase la Figura 4-3. Es básicamente la misma que la Figura 4-1, a excepción de que he añadido un par de flechas a las líneas de asociación. Estas líneas indican **navegabilidad.**

En un modelo de especificación, esto indicaría que un Pedido tiene la responsabilidad de decir a qué Cliente corresponde, pero un Cliente no tiene la capacidad correspondiente para decir cuáles son los pedidos que tiene. En lugar de responsabilidades simétricas, aquí tenemos responsabilidades de un solo lado de la línea. En un diagrama de implementación se indicaría qué Pedido contiene un apuntador a Cliente pero Cliente no apuntaría a Pedido.

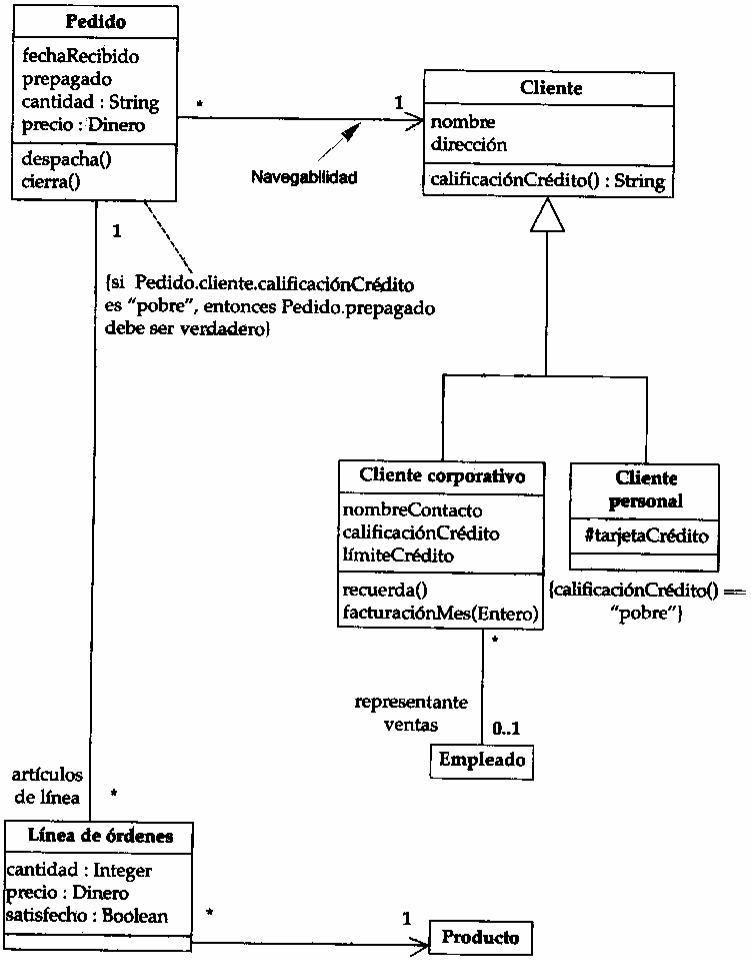


Figura 4-3: *Diagrama de clase con navegabilidades*

Figura 4-3:Diagrama de clase con navegabilidades

Como se podrá apreciar, la navegabilidad es una parte importante de los diagramas de implementación y especificación. Sin embargo, no pienso que la navegabilidad tenga utilidad en los diagramas conceptuales.

Frecuentemente verá diagramas conceptuales que primero no tendrán navegabilidades. Posteriormente, se agregarán las navegabilidades como parte del cambio a perspectivas de especificación y de implementación. Nótese también, que las navegabilidades posiblemente serán diferentes entre la especificación y la implementación.

Si existe una navegabilidad en una sola dirección, a la asociación se le llama **asociación unidireccional.** Una **asociación bidireccional** contiene navegabilidades en ambas direcciones. El UML dice que las asociaciones sin flechas significan que la navegabilidad

es desconocida o que la asociación es bidireccional. El proyecto debe definirse en el sentido de uno u otro de los significados.

Las asociaciones bidireccionales incluyen una restricción adicional, que consiste en que ambos papeles son inversos entre ellos. Esto es igual al concepto de funciones inversas en las matemáticas. En el contexto de la Figura 4-3, esto significa que cada Artículo de línea asociado con un Pedido se debe asociar con el Pedido original. De modo similar, si se toma una Línea de pedido y se busca en los artículos de línea el Pedido asociado, se deberá ver la Línea de pedido original en el conjunto. Esta cualidad se mantiene verdadera en las tres perspectivas.

Hay varias maneras de nombrar las asociaciones. A los modeladores de datos tradicionales les gusta denominar a una asociación con un verbo, de modo que se pueda emplear la relación en una oración. La mayoría de los modeladores de objetos prefieren valerse de sustantivos para denominar uno u otro papel, pues esta forma corresponde mejor a las responsabilidades y operaciones.

Algunos les dan nombres a todas las asociaciones. Yo prefiero nombrar las asociaciones sólo cuando mejora la comprensión. He visto demasiadas asociaciones con nombres como "tiene" o "se relaciona con". Si no existe un nombre para el papel, considero que su nombre es el de la clase señalada, como indiqué antes.

Arriba

Atributos

Los **atributos** son muy parecidos a las asociaciones.

Desde un nivel conceptual, el atributo de nombre de un Cliente indica que los Clientes tienen nombres. Desde el nivel de especificación, este atributo indica que un objeto Cliente puede decir su nombre y tiene algún modo de establecer un nombre. En el nivel de implementación, un Cliente tiene un **campo** (también llamado variable de instancia o miembro de datos) para su nombre.

Dependiendo del detalle del diagrama, la notación de un atributo puede mostrar el nombre, el tipo y el valor predeterminado de un atributo (la sintaxis del UML *es visibilidad nombre: tipo = valor por omisión*, donde la *visibilidad* es igual que la de las Operaciones las cuales se describirán en la sección siguiente).

Por lo tanto, ¿cuál es la diferencia entre un atributo y una asociación?

Desde la perspectiva conceptual, no hay diferencia; un atributo sólo lleva consigo otro tipo de notación, la cual puede servir si se estima conveniente. Los atributos siempre son de valor único. Por lo general, los diagramas no indican si los atributos son opcionales u obligatorios (aunque, hablando estrictamente, deberían indicado).

La diferencia se da en los niveles de especificación y de implementación. Los atributos implican navegabilidad sólo del tipo al atributo. Es más, queda implícito que el tipo contiene únicamente su propia copia del objeto de atributo, lo que implica que cualquier tipo que se emplee como atributo tendrá un valor más que una semántica de referencia.

Hablaré sobre el valor y los tipos referenciales más adelante. Por el momento, lo mejor es considerar que los atributos son clases simples y pequeñas, tales como cadenas, fechas, objetos de moneda y valores no objeto, como int y real.

Arriba 

Operaciones

Las **operaciones** son los procesos que una clase sabe llevar a cabo. Evidentemente, corresponden a los métodos sobre una clase. En el nivel de especificación, las operaciones corresponden a los métodos públicos sobre un tipo. En general, no se muestran aquellas operaciones que simplemente manipulan atributos, ya que por lo común, se pueden inferir. Sin embargo, tal vez sea necesario indicar si un **atributo dado es de sólo lectura** o está **inmutable congelado** (esto es, que su valor nunca cambia). En el modelo de implementación, se podrían mostrar también las operaciones privadas y protegidas.

La sintaxis del UML completa para las operaciones es

visibilidad nombre (lista-de-parámetros) : expresión-tipo-de-dato-a-regresar {cadena-de-propiedades} .

donde

- *visibilidad* es + (public), # (protected), o- (private)
- *nombre* es una cadena de caracteres (string)
- *lista-de-parámetros* contiene argumentos (opcionales) cuya sintaxis, es la misma que la de los atributos
- *expresión-tipo-de-dato-a-regresar* es una especificación opcional dependiente del lenguaje
- cadena-de-propiedades indica valores de propiedad que se aplican a la operación dada

Un ejemplo de operación sería: + *últimaCantidadDe (valorTipoFenómeno) : Cantidad*

Dentro de los modelos conceptuales, las operaciones no deben tratar de especificar la interfaz de una clase. En lugar de ello, deberán indicar las principales responsabilidades de dicha clase, usando tal vez un par de palabras que sinteticen una **responsabilidad de CRC.**

Con frecuencia encuentro útil hacer la distinción entre aquellas operaciones que cambian el estado de una clase y aquellas que no lo hacen. Una **consulta** es una operación que obtiene un valor de una clase sin que cambie el estado observable de tal clase. El **estado observable de un objeto** es el estado que se puede determinar a partir de sus consultas asociadas.

Considérese un objeto de Cuenta que calcula su balance a partir de una lista de entradas. Para mejorar el desempeño, Cuenta puede poner en un campo caché el resultado del cálculo del balance, para consultas futuras. Por tanto, si el caché está vacío, la primera vez que se llama a la operación "balance", pondrá el resultado en el campo caché. La operación "balance" cambia así el estado real del objeto Cuenta, pero no su estado observable, pues todas las consultas devuelven el mismo valor, esté o no lleno el campo caché. Las operaciones que sí cambian el estado observable de un objeto se llaman **modificadores.**

Considero útil tener perfectamente clara la diferencia entre consultas y modificadores. Las consultas se pueden ejecutar en cualquier orden, pero la secuencia de los modificadores es más importante. Yo tengo como política evitar que los modificadores devuelvan valores, con el fin de mantenerlos separados.

Otros términos que algunas veces se presentan son: métodos de obtención y métodos de establecimiento. El **método de obtención** (*getting*) es un método que devuelve el valor de un campo (sin hacer nada más). El **método de establecimiento** (*setting*) pone un valor en un campo (y nada más). Desde afuera, el cliente no debe poder saber si una consulta es un método de obtención ni si un modificador es un método de establecimiento. El conocimiento sobre los métodos de obtención y establecimiento está contenido completamente dentro de la clase.

Otra distinción es la que se da entre operación y método. Una **operación** es algo que se invoca sobre un objeto (la llamada de procedimiento), mientras que un **método** es el cuerpo del procedimiento. Los dos son diferentes cuando se tiene polimorfismo. Si se tiene un supertipo con tres subtipos, cada uno de los cuales suplanta la operación "*foo*" del supertipo, entonces lo que hay es una operación y cuatro métodos que la implementan.

Es muy común que operación y método se empleen indistintamente, pero hay veces en que es necesario precisar la diferencia. En algunas ocasiones, la gente distingue entre una y otro mediante los términos llamada a un método, o declaración de método (en lugar de operación), y cuerpo del método.

Los lenguajes tienen sus propias convenciones para los nombres. En C ++, las operaciones se llaman funciones miembro; En Smalltalk, operaciones de métodos. C ++ también emplea el término miembros de una clase para denominar las operaciones y métodos de una clase.

Arriba

Generalización

Un ejemplo característico de **generalización** comprende al personal y las Corporaciones clientes de una empresa. Unos y otros tienen diferencias, pero también muchas similitudes. Las similitudes se pueden colocar en una clase general Cliente (el **supertipo**) con los subtipos siguientes: Cliente personal y Cliente corporativo.

Este fenómeno también está sujeto a diversas interpretaciones, según el nivel de modelado. Por ejemplo, conceptualmente, podemos decir que el Cliente corporativo es un **subtipo** de Cliente, si todas las instancias de Cliente corporativo, también son, por definición, instancias de Cliente. Entonces, un Cliente corporativo es un tipo especial de Cliente. El concepto clave es que todo lo que digamos de un Cliente (asociaciones, atributos, operaciones) también vale para un Cliente corporativo.

En un modelo de especificación, la generalización significa que la **interfaz** del subtipo debe incluir todos los elementos de la interfaz del supertipo. Se dice, entonces, que la interfaz del subtipo se conforma con la interfaz del supertipo.

Otra manera de considerar esto involucra el principio de la **sustituibilidad.** Yo debo poder sustituir a un Cliente corporativo con cualquier código que requiera un Cliente y todo debe operar sin problemas. En esencia, esto significa que, si escribo un código suponiendo que tengo un Cliente, puedo entonces usar con toda libertad cualquier subtipo de Cliente. El Cliente corporativo puede responder a ciertos comandos de manera diferente a otro Cliente (por el principio del polimorfismo), pero el invocador no debe preocuparse por la diferencia.

La generalización desde la perspectiva de implementación se asocia con la herencia en los lenguajes de programación. La subclase hereda todos los métodos y campos de la superclase y puede suplantarlos.

El punto clave aquí es la diferencia entre generalización desde la perspectiva de la especificación (subtipificación o herencia de interfaces) y desde la perspectiva de la implementación (subclasificación o herencia de implementaciones). La **subclasificación** (formación de subclases) es una manera de implementar la subtipificación. La **subtipificación** (formación de subtipos) también se puede implementar mediante delegación.

En el caso de estas dos formas de generalización, se deberá asegurar siempre que también tenga validez la **generalización conceptual.** He encontrado que, si no se hace lo anterior,

se puede uno ver en problemas, debido a que la generalización no es estable cuando hay necesidad de efectuar cambios posteriores.

En algunas ocasiones aparecen casos en los que un subtipo tiene la misma interfaz que el supertipo, pero el subtipo implementa sus operaciones de modo diferente. Si éste es el caso, se podría decidir no mostrar el subtipo en el diagrama de perspectiva de especificación. Así lo hago con frecuencia si para los usuarios de la clase es interesante la existencia de los tipos, pero no lo hago cuando los subtipos varían únicamente debido a razones internas de implementación.

Arriba

Reglas de restricción

Una buena parte de lo que se hace cuando se dibuja un diagrama de clase es indicar las condiciones limitantes o restricciones.

La Figura 4-3 indica que un Pedido únicamente puede ser colocado por un solo Cliente. El diagrama implica también que Artículos de línea se considera por separado: digamos 40 adminículos de color marrón, 40 adminículos azules y 40 adminículos rojos, y no 40 artefactos rojos, azules y marrones. El diagrama dice, además, que los Clientes corporativos tienen límites de crédito, pero que los Clientes personales no los tienen.

Los artificios básicos de la asociación, el atributo y la generalización, colaboran de manera significativa con la especificación de condiciones importantes, pero no pueden indicarlas todas. Estas restricciones aún necesitan ser captadas; el diagrama de clase es un lugar adecuado para hacerlo.

El UML no define una sintaxis estricta para describir las restricciones, aparte de ponerlas entre llaves ({ }). Yo prefiero escribir en lenguaje informal, para simplificar su lectura. Puede usarse también un enunciado más formal, como un cálculo de predicados o alguna derivada. Otra alternativa es escribir un fragmento de código de programación.

Idealmente, las reglas se deberían implementar como afirmaciones en el lenguaje de programación. Éstas se corresponden con el concepto de invariantes del **Diseño por contrato** En lo personal, soy partidario de crear un método *revisalnvariante* e invocarlo desde algún código de depuración que ayude a comprobar los invariantes.

Arriba

Cuándo emplear los diagramas de clases

Los diagramas de clase son la columna vertebral de casi todos los métodos OO, por lo cual usted descubrirá que se emplean todo el tiempo. Este capítulo ha abarcado los conceptos básicos; el capítulo siguiente, analizará muchos de los conceptos más avanzados.

El problema con los diagramas de clase es que son tan ricos que pueden resultar abrumadores. A continuación, damos algunos consejos breves

- No trate de usar todas las anotaciones a su disposición. Comience con el material sencillo de este capítulo: clases, asociaciones, atributos y generalización. Introduzca otras anotaciones más avanzadas sólo cuando las necesite.
- Ajuste la perspectiva desde la cual se dibujan los modelos a la etapa del proyecto.
    - Si se está en la etapa del análisis, dibuje modelos conceptuales.
    - Cuando se trabaje con software, céntrese en los modelos de especificación.
    - Dibuje modelos de implementación sólo cuando se esté ilustrando una técnica de implementación en particular.
- No dibuje modelos para todo; por el contrario, céntrese en las áreas clave. Es mejor usar y mantener al día unos cuantos diseños que tener muchos modelos olvidados y obsoletos.

El principal peligro con los diagramas de clase es que se puede quedar empantanado muy pronto en los detalles de la implementación. Para contrarrestar esto, céntrese en las perspectivas conceptual y de especificación. Si cae en estos problemas, encontrará muy útiles las tarjetas de CRC

Arriba

Para mayor información

Mi consejo sobre otras fuentes depende de si usted prefiere una perspectiva de implementación o conceptual. Para la perspectiva de implementación, es recomendable Booch (1994); Para una perspectiva conceptual son recomendables los libros de "fundamentos" de Martin y Adell (1994). Una vez que haya leído su primera selección, lea la otra, pues ambas perspectivas son importantes. Después, cualquier buen libro sobre OO le dará algunos puntos de vista interesantes. Tengo una particular predilección por el de Cook y Daniels (1994), debido a su manejo de las perspectivas y a la formalidad que introducen los autores.

Arriba

Índice

- Asociaciones
- Las asociaciones
- asociación unidireccional (Las asociaciones)
- asociación bidireccional (Las asociaciones)
- atributos
- atributo dado es de sólo lectura (atribuitos)
- campo (atributos)
- Conceptual (Perspectivas)
- congelado (atribuitos)
- conjunto de objetos relacionados (Las asociaciones)
- consulta
- Diagrama de clase
- Diseño por contrato
- Destino (Las asociaciones, papeles)
- Especificación (Perspectivas)
- estado observable de un objeto
- generalización
- generalización conceptual(generalización)
- Implementación (Perspectivas)
- inmutable (atribuitos)
- interfaz (generalización)
- método
- métodos de consulta(Las asociaciones)
- método de obtención
- método de establecimiento
- modelo de implementación(Las asociaciones)
- modificadores de un objeto
- Multiplicidad (Las asociaciones, papeles)
- Navegabilidad (Las asociaciones)
- operación
- operaciones
- origen (Las asociaciones, papeles)
- perspectiva conceptual (Las asociaciones)
- papeles (Las asociaciones)
- perspectiva de la especificación (Las asociaciones)
- relaciones de un solo valor (Las asociaciones)
- Tarjetas de CRC
- Subtipos

- Terminología de diagramas de clase
- subclasificación (generalización)
- subtipificación (generalización)
- subtipo (generalización)
- supertipo (generalización)
- sustituibilidad (generalización)

Arriba

Índice de Contenidos

- Diagramas de clase: conceptos avanzados
  -
    - Resumen
    - Mapa General
    - Índice de Contenidos
    - Introducción
    - Los estereotipos
    - Clasificación múltiple y dinámica
    - Agregación y composición
    - Asociaciones y Atributos Derivados
    - Interfaces y clases abstractas
    - Objetos de referencia y objetos de valor
    - Colecciones para relaciones de valor múltiple
    - Congelado
    - Clasificación y generalización
    - Asociaciones calificadas
    - Clase de asociación
    - Clase con parámetro
    - La visibilidad
    - Características del alcance de clase
    - Para mayor información
    - Índice
    - Referencia Bibliográfica
    - Pie del Documento

Introducción

Los conceptos descritos en el capítulo anterior corresponden a las notaciones fundamentales de los diagramas de clase. Éstos son los que usted necesita comprender y dominar primero, pues significarán el 90% del esfuerzo que se invertirá en la construcción de los diagramas de clase.

La técnica de diagramas de clase, sin embargo, ha generado decenas de nuevas notaciones para otros conceptos. En realidad, éstos no se emplean con frecuencia, pero hay ocasiones en las que resultan muy útiles.

A continuación las explicaré una por una y señalaré algunas de las consideraciones sobre su uso. No obstante, recuerde que son opcionales y que hay mucha gente que ha obtenido grandes ventajas de los diagramas de clase sin haber utilizado estas características adicionales.

Tal vez encuentre el presente capítulo un poco denso. Sin embargo, tengo una buena noticia al respecto, y es que este capítulo puede usted saltado completo, sin ningún problema, en la primera lectura que haga del libro, y volver a él más tarde.

Arriba 

Los estereotipos

La idea de los **estereotipos** fue acuñada por Rebecca Wirfs-Brock. El concepto ha sido adoptado con entusiasmo por los inventores del UML, aunque de un modo que en realidad no significa lo mismo. Sin embargo, ambas ideas son valiosas.

La idea original de un estereotipo se refería a una clasificación de alto nivel de un objeto que diera alguna indicación del tipo de objeto que era. Un ejemplo es la diferencia entre un "controlador" y un "coordinador".

Con frecuencia se encuentran diseños OO en los que una clase parece hacer todo el trabajo, muchas veces por medio de un gran método "hazlo" (*doIt*), mientras que las demás clases no hacen otra cosa más que encapsular datos. Éste es un diseño pobre, pues significa que el controlador es muy complejo y difícil de manejar.

Para mejorar esta situación, se traslada el comportamiento del controlador a los objetos de datos relativamente tontos, de modo que éstos se vuelven más inteligentes y adquieren responsabilidades de más alto nivel. Así, el controlador se convierte en un coordinador. El coordinador es el encargado de disparar tareas en una secuencia particular, pero otros objetos son los que saben cómo desempeñadas.

La esencia del estereotipo consiste en que sugiere ciertas responsabilidades generales de una clase. El UML ha adoptado este concepto y lo ha convertido en un mecanismo general de extensión del lenguaje mismo.

En su trabajo original (1994), Jacobson clasifica todas las clases de un sistema en tres categorías: objetos de interfaz, objetos de control y objetos de entidad (sus objetos de control, cuando están bien diseñados, son parecidos a los coordinadores de Wirfs-Brock). Jacobson sugirió reglas para la comunicación entre estos tipos de clases y les dio a cada una de ellas un icono diferente. Esta distinción no es una parte medular del UML. Al contrario, este tipo de clases constituye en realidad estereotipos de clases; de hecho, son estereotipos, en el sentido que Wirfs-Brock le da al término.

Los estereotipos generalmente se indican en el texto entre comillas francesas («objeto de control»), pero también se pueden mostrar definiendo un icono para el estereotipo. De lo que se trata es de que, si no se está empleando el enfoque de Jacobson, se pueden olvidar los estereotipos. Pero si quiere trabajar con dicho enfoque, se pueden definir los estereotipos y las reglas para su uso.

Muchas extensiones del núcleo de UML se pueden describir como una colección de estereotipos. En los diagramas de clase, pueden ser estereotipos de clases, asociaciones o generalizaciones. Puede pensarse en los estereotipos como subtipos de los tipos Clase, Asociación y Generalización en el metamodelo.

He observado que la gente que trabaja con el UML tiende a confundir las restricciones, es decir, las condiciones limitantes con los estereotipos. Si se marca una **clase como abstracta,** ¿se trata de una restricción o de un estereotipo? Los documentos oficiales actuales llaman a esto restricción, pero hay que estar conscientes de que se hace un uso confuso de una y otro. Esto no es de sorprender, ya que los subtipos son con frecuencia más limitados que los supertipos.

Arriba

Clasificación múltiple y dinámica

La **clasificación** se refiere a la relación entre un objeto y su tipo.

La mayor parte de los métodos hacen ciertas suposiciones sobre este tipo de relación (premisas que están presentes también en los principales lenguajes de programación OO). Estas premisas fueron cuestionadas por Jim Odell, quien las consideró demasiado restrictivas para el modelado conceptual. Las suposiciones son para una clasificación simple y estática de los objetos; Odell sugiere emplear una clasificación múltiple y dinámica de los objetos en los modelos conceptuales.

En la **clasificación simple,** un objeto pertenece a un solo tipo, que puede haber heredado de supertipos. En la **clasificación múltiple,** un objeto puede ser descrito por varios tipos que no están conectados necesariamente por medio de herencia.

Obsérvese que la clasificación múltiple es diferente de la **herencia múltiple.** La herencia múltiple plantea que un tipo puede tener muchos supertipos, pero que se debe definir un solo tipo por cada objeto. La clasificación múltiple permite varios tipos para un objeto sin definir un tipo específico para tal fin.

Como ejemplo, considere una persona subtipificada como hombre o mujer, doctor o enfermera, paciente o no (véase la Figura 5-1). La clasificación múltiple permite asignar cualquiera de estos tipos a un objeto en cualquier combinación posible, sin necesidad de que se definan tipos para todas las combinaciones legales.

Si se emplea la clasificación múltiple, habrá de asegurarse que haya quedado claro cuáles son las combinaciones legales. Esto se hace etiquetando una línea de generalización con un discriminador, el cual es una indicación de la base de la subtipificación. Varios subtipos pueden compartir el mismo discriminador. Todos los subtipos con el mismo discriminador están desunidos, es decir, cualquier instancia del supertipo puede ser una instancia de sólo uno de los subtipos, dentro del discriminador. Una buena convención es que todas las subclases que emplean un discriminador desemboquen en un triángulo, como se muestra en la Figura 5-1. Se pueden tener, como alternativa, varias flechas con la misma etiqueta.

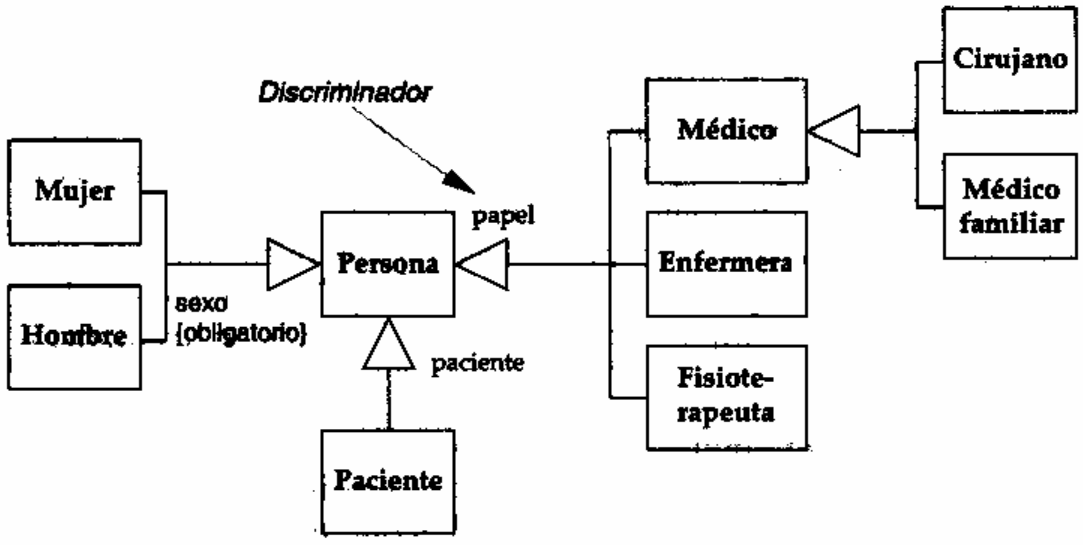


Figura 5-1: *Clasificación múltiple*

Figura 5-1: Clasificación múltiple

Una restricción útil (pero no estándar en el UML) consiste en marcar al discriminador como {obligatorio}. Esto significa que cualquier instancia de la superclase debe ser una instancia de una de las subclases del grupo (por tanto, la superclase es abstracta).

Para ilustrar, obsérvense las siguientes combinaciones legales de subtipos en el diagrama: (Mujer, Paciente, Enfermera); (Hombre, Fisioterapeuta); (Mujer, Paciente); y (Mujer, Médico, Cirujano). Obsérvense también que las combinaciones como (Paciente, Médico) y (Hombre, Médico, Enfermera) son ilegales: la primera, porque no incluye un tipo del discriminador Sexo {obligatorio}; la última, porque contiene dos tipos del discriminador "Papel". La clasificación simple, por definición, corresponde a un solo discriminador no etiquetado.

Otra pregunta que cabe plantearse es si un objeto puede cambiar de tipo. Un buen ejemplo para contestada es una cuenta bancaria. Cuando se sobregira la cuenta, cambia sustancialmente su conducta; específicamente se suplantan varias operaciones (incluyendo la de "retiro" y la de "cierre").

La **clasificación dinámica** permite a los objetos cambiar de tipo dentro de la estructura de subtipificación; la **clasificación estática,** no. En la clasificación estática se hace la distinción entre tipos y estados; en la clasificación dinámica se combinan estos dos conceptos.

¿Debería usarse la clasificación múltiple y dinámica? Considero que es útil para el modelado conceptual. Se puede hacer con modelado por especificaciones, pero conviene sentirse cómodo con las técnicas necesarias para realizado. El truco consiste en implementado de manera que parezca lo mismo que subclasificar desde la interfaz, de tal suerte que el usuario de una clase no sepa qué implementación se está usando. Sin embargo, como en la mayoría de estas cosas, la elección depende de las circunstancias, así que se deberá aplicar lo que se juzgue mejor. La transformación de una interfaz múltiple y dinámica a una implementación estática única bien puede acarrear más problemas de los que resolvería.

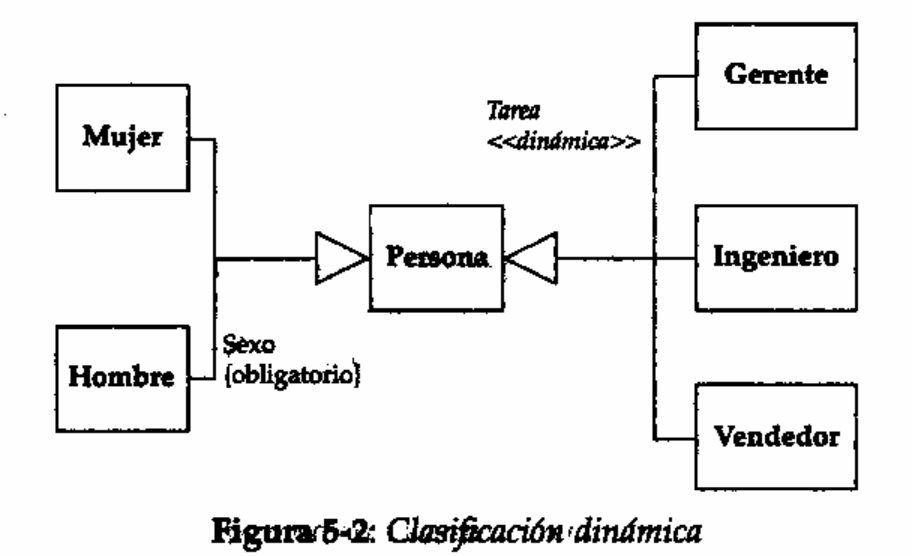


Figura 5-2: Clasificación dinámica

Figura 5-2:Clasificación dinámica

La figura 5-2 muestra un ejemplo del empleo de la clasificación dinámica para el trabajo de una persona, que, por supuesto, puede cambiar. Esto puede ser lo apropiado, pero los subtipos necesitarían un comportamiento adicional, en lugar de ser sólo etiquetas. En estos casos, muchas veces vale la pena crear una clase separada para el trabajo y vincular a la persona con ella mediante una asociación.

Arriba

Agregación y composición

Una de las cosas más aterradoras del modelado es la **agregación.** Ésta es fácil de explicar con pocas palabras: la agregación es la relación de componentes. Es como decir que un automóvil tiene como componentes motor y ruedas. Esto suena muy bien, pero la parte difícil es determinar cuál es la diferencia entre agregación y asociación.

Peter Coad ejemplificó la agregación como la relación entre una organización y sus empleados; Jim Rumbaugh afirmó que una compañía no es la agregación de sus empleados. Cuándo los gurúes no se ponen de acuerdo, ¿qué debemos hacer? El problema es que los metodólogos no tienen una definición aceptada por todos de la diferencia entre asociación y agregación.

De hecho, muy pocos se sirven de cualquier definición formal. La cuestión práctica que nos interesa aquí es que todos manejan un concepto levemente diferente, de modo que hay que

tener precaución con este concepto, y, generalmente, prefiero evitado, a menos que el equipo del proyecto se ponga de acuerdo en un significado riguroso y útil.

El UML ofrece, además de la agregación simple, una variedad más poderosa, la cual se denomina composición. Con la **composición,** el objeto parte puede pertenecer a un todo único; es más, se espera, por lo general, que las partes vivan y mueran con el todo. Cualquier borrado del todo se extiende en cascada a las partes.

El borrado en cascada se considera con frecuencia como una parte definitoria de la agregación, pero está implícita en cualquier papel con una multiplicidad de [1..1]; si realmente se quiere eliminar a un Cliente, por ejemplo, habrá que aplicar este borrado en cascada a Pedido (y, por tanto, a Línea de pedido).

La Figura 5-3 muestra ejemplos de estos artificios. Las composiciones a Punto indican que cualquier instancia de Punto puede estar en un Polígono o en un Círculo, pero no en ambos. Sin embargo, una instancia de Estilo puede ser compartida por muchos Polígonos y Círculos. Es más, esto implica que el borrado de un Polígono provocaría el borrado de sus Puntos asociados, pero no del Estilo asociado.

Esta restricción (un Punto puede solamente aparecer en un solo Polígono o Círculo a la vez) no se puede expresar mediante las multiplicidades, únicamente. También implica que el punto es un objeto de valor(Vease mas adelante en este mismo capitulo). Se puede añadir una multiplicidad a un lado compuesto de la asociación, pero yo no me tomo la molestia. El rombo negro dice todo lo que hay que decir.

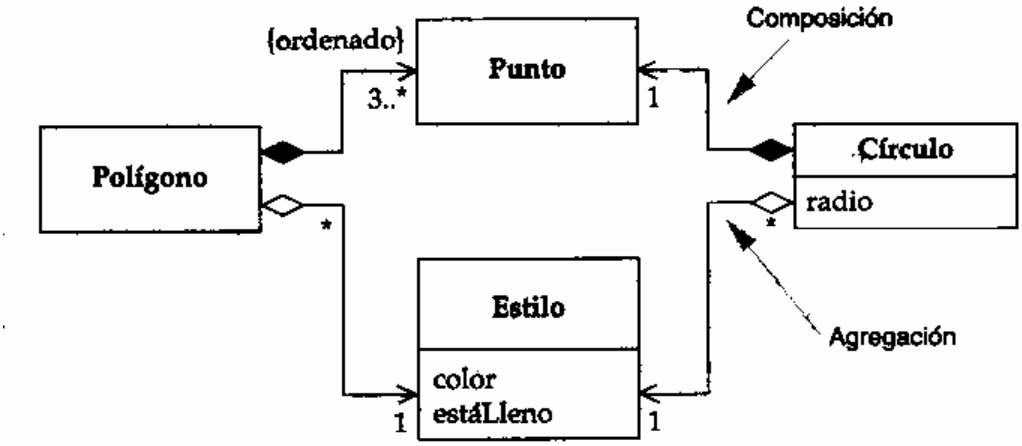


Figura 5-3: *Agregación y composición*

Figura 5-3:Agregación y composición

La figura 5-4 muestra otra notación para la composición. En este caso, se pone el componente dentro del todo. El nombre de la clase del componente no está en negritas y se

escribe en la forma nombrepapel: Clase nombre. Además, se pone la multiplicidad en la esquina superior derecha.

Las diferentes notaciones de composición operan en situaciones diferentes. Hay un par más, aunque la variedad de notaciones de composición ofrecida por el UML es más bien abrumadora. Obsérvese que estas variaciones sólo pueden servir para la composición, no para la agregación.

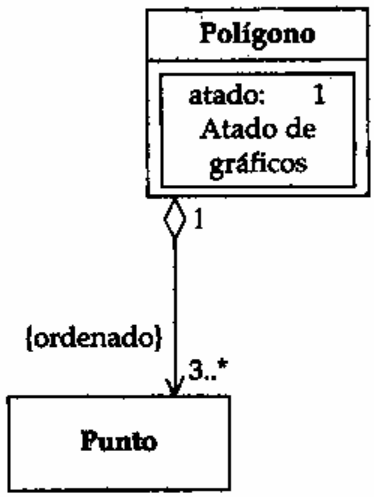


Figura 5-4: *Notación alterna para la composición*

Figura 5-4:Notación alterna para la composición

Arriba

Asociaciones y Atributos Derivados

Las **asociaciones derivadas** y los **atributos derivados** son aquellos que se pueden calcular a partir de otras asociaciones y atributos, respectivamente, de un diagrama de clase. Por ejemplo, un atributo de edad de una Persona se puede derivar si se conoce su fecha de nacimiento.

Cada perspectiva contribuye con su propia interpretación de las características derivadas de los diagramas de clase. La más crítica de éstas se relaciona con la perspectiva de especificación. Desde este ángulo, es importante comprender que las características derivadas indican una restricción entre valores y no una declaración de lo que se calcula y lo que se almacena.

La figura 5-5 muestra una estructura jerárquica de cuentas dibujada desde una perspectiva de especificación. El modelo utiliza el patrón Compuesto.

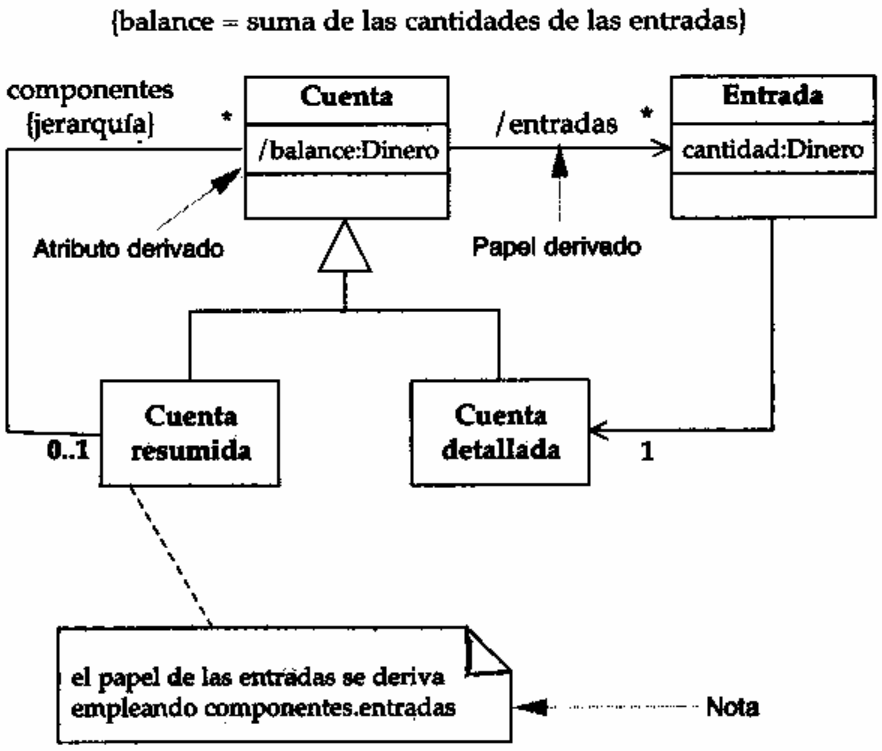


Figura 5-5: *Asociaciones y atributos derivados*

Figura 5-5:Asociaciones y atributos derivados

Obsérvese lo siguiente:

- Los objetos de entrada están conectados a Cuentas detalladas.
- El balance de una Cuenta se calcula como la suma de las cantidades de Entrada.
- Las entradas de una Cuenta resumida son las entradas de sus componentes, determinados de manera recurrente.

Debido a que la Figura 5-5 ilustra un modelo de especificación, no indica que las clases Cuentas no contienen campos para hacer balances; bien podría estar presente tal caché, pero está oculto para los clientes de la clase Cuenta.

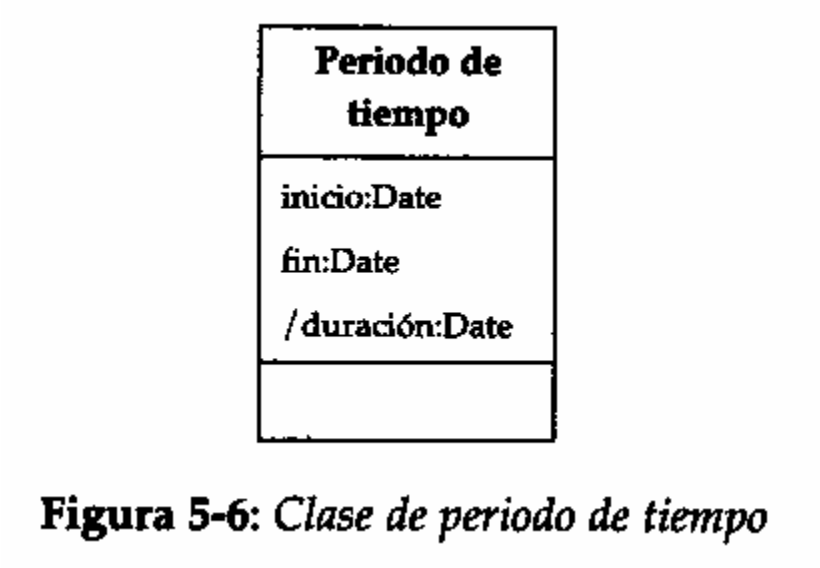


Figura 5-6: *Clase de periodo de tiempo*

Figura 5-6: Clase de periodo de tiempo

Puedo ilustrar cómo los elementos derivados indican restricciones de una clase titulada Periodo de tiempo (véase la Figura 5-6). Si éste es un diagrama de especificación, entonces, aunque sugiera que inicio y fin están almacenados y duración se calcula, el programador puede, de hecho, implementar esta clase de cualquier modo que mantenga dicha conducta externa. Por ejemplo, es perfectamente aceptable almacenar inicio y duración y calcular fin.

En los diagramas de implementación, los valores derivados son valiosos para anotar campos que se emplean como cachés por razones de desempeño. Al marcarlos y registrar la derivación del caché, es más fácil ver explícitamente lo que hace éste. Con frecuencia, refuerzo esto en el código indicando la palabra "caché" en tal campo (por ejemplo, cachéBalance) .

En los diagramas conceptuales, utilizo marcadores derivados para que me recuerden dónde existen tales derivaciones y confirmar, con los expertos del dominio, que existen las derivaciones. Entonces, éstas se correlacionan con su empleo en los diagramas de especificación.

En los mundos del OMT y de Odell, las asociaciones derivadas se indican con una diagonal en la línea de asociación. Este uso no es parte del UML, aunque, no obstante, confieso que lo utilizo. Me parece más claro, en particular cuando no le doy nombre a la asociación.

Arriba

Interfaces y clases abstractas

Una de las grandes cualidades del desarrollo orientado a objetos es que se pueden variar las interfaces de las clases, independientemente de la implementación. Gran parte del poder del desarrollo de objetos surge de esta propiedad. Muy pocos, sin embargo, hacen un buen uso de ella.

Los lenguajes de programación usan un solo artificio, la clase, que contiene tanto la interfaz como la implementación. Cuando se subclasifica, se heredan ambas. Se recurre muy poco a utilizar la interfaz como un artificio separado, lo cual es una verdadera lástima.

Una interfaz pura (como la de Java) es una clase sin implementación y, por tanto, tiene declaraciones de operaciones, pero no cuerpos de método ni campos. Con frecuencia, las interfaces se declaran por medio de clases abstractas. Dichas clases pueden proporcionar alguna realización, pero con frecuencia se usan principalmente para declarar una interfaz. La cuestión es que la subclasificación (o cualquier otro mecanismo) proporcionará la implementación, pero los clientes sólo verán la interfaz y no la realización.

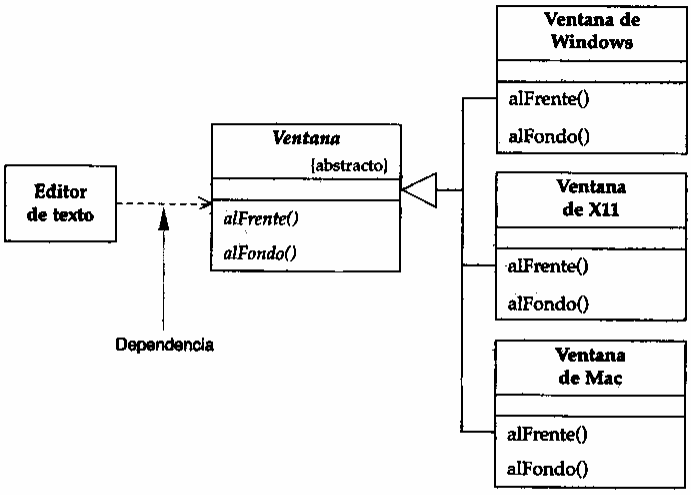


Figura 5-7: *Ventana como clase abstracta*

Figura 5-7:Ventana como clase abstracta

El editor de texto representado en la Figura 5-7 es un ejemplo típico de lo anterior. Para permitir que el editor sea independiente de la plataforma, definimos una clase abstracta Ventana, independiente de la plataforma. Esta clase no tiene operaciones; sólo define una interfaz para el editor de texto. Las subclases específicas de la plataforma se pueden emplear como se desee.

Si se tiene una clase o método abstractos, la convención en el UML es poner con cursivas el nombre del elemento abstracto. La restricción {abstracto} también puede utilizarse (y también en sustitución). Por mi parte, manejo {abstracto} en los pizarrones, porque no

puedo escribir el texto en cursivas. Sin embargo, teniendo una herramienta de diagramación, prefiero la elegancia de las cursivas.

Pero la subclasificación no es la única manera de hacer lo anterior. Java ofrece una interfaz específica y el compilador revisa que la clase ofrezca una implementación para todas las operaciones definidas para dicha interfaz.

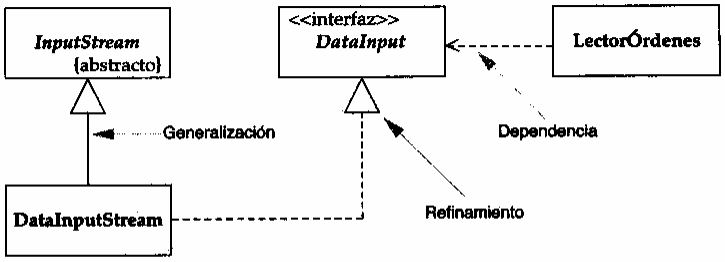


Figura 5-8: *Interfaces y clase abstracta: un ejemplo de Java*

Figura 5-8:Interfaces y clase abstracta: un ejemplo de Java

En la Figura 5-8, vemos InputStream, DataInput y DataInputStream (definidos en el archivo java.io estándar). InputStream es una clase abstracta; DataInput es una interfaz.

Alguna clase cliente, digamos, LectorÓrdenes necesita utilizar la funcionalidad de DataInput. La clase de DataInputStream implementa las interfaces DataInput e InputStream y es una subclase de esta última.

El vínculo entre DataInputStream y DataInput es una relación de refinamiento. El **refinamiento** es un término general que se emplea en el UML para indicar un mayor nivel de detalle. Puede usarse para la implementación de interfaces o algún otro fin (véanse los libros de los tres amigos para mayores detalles). El refinamiento deliberadamente es similar a la generalización.

En un modelo de especificación, tanto la subclasificación como el refinamiento se representarían como una subtipificación; la diferencia entre el refinamiento y la generalización es válida sólo en los modelos de implementación. Algunos modeladores prefieren usar «tipo» en lugar de «interfaz». En mi opinión, soy partidario de «interfaz», pues así queda explícito el papel de la interfaz.

El vínculo entre LectorÓrdenes y DataInput es una dependencia. Muestra que LectorÓrdenes usa la interfaz de DataInput con algún fin. En esencia, una dependencia indica que, si la interfaz de DataInput cambia, también tiene que cambiar el LectorÓrdenes. Uno de los objetivos del desarrollo es el de mantener a un mínimo las dependencias, de modo que los efectos de los cambios también sean mínimos.

La Figura 5-9 muestra una notación alterna más compacta. En ella, las interfaces están representadas por pequeños círculos llamados con frecuencia paletas (*lollipops*) que surgen de las clases que las implementan.

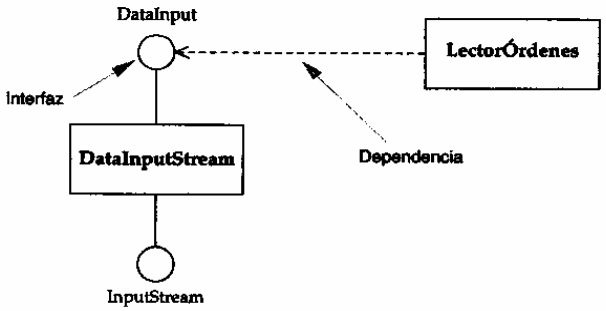


Figura 5-9: *Notación de paletas para representar interfaces*

Figura 5-9: Notación de paletas para representar interfaces

No existe diferencia alguna entre el refinamiento de una interfaz y la subclasificación de una clase abstracta.

Las clases abstractas y las interfaces son similares, aunque existe una diferencia entre ellas. Ambas le permiten definir una interfaz y diferir su realización para más tarde. Por otra parte, la clase abstracta permite añadir la implementación de algunos de los métodos; en cambio, una interfaz obliga a diferir la definición de todos los métodos.

Arriba

Objetos de referencia y objetos de valor

Una de las cosas comunes que se dicen de los objetos es que tienen identidad. Esto es cierto, pero el asunto no es tan simple como parece. En la práctica, usted se percata de que la identidad es importante para los objetos de referencia, pero no tanto para los objetos de valor.

Los **objetos de referencia** son cosas como Cliente. La identidad aquí es muy importante, porque usualmente se quiere que un solo objeto de software designe a un cliente del mundo real. Cualquier objeto que se refiera al objeto Cliente lo hará por medio de una referencia o un apuntador; todos los objetos que se refieran a este Cliente se referirán al mismo objeto de software. De esta manera, los cambios en un Cliente estarán disponibles para todos los usuarios del Cliente.

Si se tienen dos referencias de un Cliente y se desea saber si son iguales, por lo común se comparan sus identidades. Las copias probablemente no estén permitidas y, si lo están, tenderán a hacerse en raras ocasiones, tal vez para propósitos de respaldo o para su réplica en una red. Si se hacen copias, habrá que determinar la manera de sincronizar los cambios.

Los **objetos de valor** son cosas como Fecha. Con frecuencia, se tienen varios objetos de valor que representan al mismo objeto en la realidad. Por ejemplo, es normal tener a cientos de objetos que designen el 1 de enero de 1997. Todas éstas son copias intercambiables. A menudo se crean y destruyen fechas nuevas.

Si se tienen dos fechas y se quiere saber si son iguales, no se ven sus identidades; se ven los valores que representan. Esto, por lo general, significa que hay que escribir un operador de prueba de igualdad, el cual, hablando de fechas, efectuará pruebas sobre año, mes y día (o cualquiera que sea la representación interna). Usualmente, cada objeto que se refiera al 1° de enero de 1997 tendrá su propio objeto dedicado, pero en ocasiones se pueden tener fechas combinadas.

Los objetos de valor deben ser **objetos de inmutables.** En otras palabras, no se debe poder tomar un objeto con fecha del 1 de enero de 1997 y cambiarlo al 2 de enero de 1997. En cambio, se deberá crear un nuevo objeto 2 de enero de 1997 y vinculado al primer objeto. La razón de esto es que si la fecha fuera compartida, se actualizaría la fecha de otro objeto de manera impredecible.

En C ++ esto no constituye un problema, pues se hace un gran esfuerzo por no compartir fechas; el compartir objetos de valor conduce a problemas de administración de memoria. En lugar de eso, se puede suplantar la asignación para hacer copias. En los ambientes de memoria administrada, tales como Java, esto es más importante, en especial porque, en Java, las fechas no son inmutables. Como regla empírica, no cambie un objeto de valor.

En el pasado, la diferencia entre los objetos de referencia y los objetos de valor era más clara. Los objetos de valor eran los valores interconstruidos del sistema de tipos. En la actualidad, se puede extender el sistema de tipos con clases propias del usuario, por lo que esta cuestión requiere de mayor análisis. En el UML, los atributos se usan generalmente para objetos de valor y las asociaciones para objetos de referencia. También se puede manejar composición para objetos de valor.

No considero que la diferencia entre objetos de referencia y objetos de valor sea de utilidad en los modelos conceptuales. No hay diferencia entre las dos construcciones desde la perspectiva conceptual. Si represento un vínculo con un objeto de valor mediante una asociación, marco como [*] la multiplicidad del papel desde el valor a su usuario, a menos que exista una regla de unicidad (como, digamos, un número de secuencia).

Arriba 

Colecciones para relaciones de valor múltiple

Una **relación de valor múltiple** (*multi-valued role*) es aquella cuyo límite más alto de multiplicidad es mayor que 1 (por ejemplo, *). La convención usual es que las relaciones de valor múltiple se consideran como conjuntos. No hay un ordenamiento para los objetos destino y ningún objeto destino aparece más de una vez en la relación. Sin embargo, se pueden cambiar estas premisas al conectar una restricción.

La restricción {ordenado} implica que hay un ordenamiento para los objetos destino (esto es, los objetos destino forman una lista). Los objetos destino pueden aparecer sólo una vez en la lista.

Yo utilizo la restricción {bolsa} para indicar que los objetos destino pueden aparecer más de una vez, pero no hay un ordenamiento (si quisiera un ordenamiento y varias apariciones, indicaría {ordenado bolsa}, aunque aún no he tenido necesidad de hacerlo). También manejo la restricción {jerarquía} para indicar que los objetos destino forman una jerarquía.

Arriba 

Congelado

Congelado (*frozen*) es una restricción que, según la define el UML, se aplica a los papeles pero también puede aplicarse a los atributos y a las clases.

En un papel o en un atributo, congelado indica que sus valores no pueden cambiar durante la vida del objeto origen. El valor debe establecerse en el momento de la creación del objeto y nunca puede cambiar. El valor inicial puede ser nulo. Ciertamente, si éste es el valor en el momento de la construcción del objeto, así seguirá, mientras el objeto viva. Esto implica que debe haber un argumento para este valor en un constructor y que no debe haber ninguna operación que lo actualice.

Cuando se aplica a una clase, congelado indica que todas las funciones y atributos asociados con esa clase están congelados.

Congelado no es igual que la restricción sólo-lectura. La restricción sólo-lectura implica que un valor no puede cambiarse directamente, sino que puede cambiar debido a la modificación de algún otro valor. Por ejemplo, si una persona tiene una fecha de nacimiento y una edad, entonces la edad puede ser de sólo lectura, pero no congelado. Marco "congelamiento" mediante la restricción {congelado} y los valores de sólo lectura con {sólo lectura}.

Si usted tiene la intención de "congelar" algo, tenga en cuenta que las personas cometen errores. En el software modelamos lo que conocemos sobre el mundo, no cómo es el mundo. Si modeláramos cómo es el mundo, el atributo de "fecha de nacimiento" de un objeto Persona sería inmutable, pero, en la mayoría de los casos, querríamos cambiarlo si nos percatamos de que un registro previo es incorrecto.

Arriba 

Clasificación y generalización

Con frecuencia, oigo a las personas referirse a la subtipificación como la relación de "es un". Le recomiendo muy encarecidamente tener cuidado con esta manera de pensar. El problema es que la frase "es un" puede significar cuestiones diferentes.

Considere las siguientes frases:

1. Shep es un Border Collie.
2. Un Border Collie es un Perro.
3. Los Perros son Animales.
4. Un Border Collie es una Raza.
5. El Perro es una Especie.

A continuación, intente combinar las frases anteriores. Si se combinan las frases 1 y 2, se obtiene "Shep es un Perro"; la combinación de las frases 2 y 3 da "los Border Collie son Animales". Y 1 más 2 más 3 da "Shep es un Animal" . Hasta aquí no hay problemas. Ahora, inténtese 1 con 4: "Shep es una Raza". La combinación de 2 con 5 da "un Border Collie es una Especie." Estas combinaciones ya no son tan buenas.

¿Por qué se pueden combinar algunas de estas frases y otras no? La razón es que algunas son **clasificaciones** (el objeto Shep es una instancia del tipo Border Collie) y otras son **generalizaciones** (el tipo Border Collie es un subtipo del tipo Perro). La generalización es transitiva, pero la clasificación no. Puedo combinar una clasificación seguida por una generalización, pero no puedo llevar a cabo la combinación a la inversa.

Puntualizo lo anterior para que desconfíe de la frase "es un". Su uso puede conducir al empleo inapropiado de la subclasificación y a la confusión de responsabilidades. Las frases "los Perros son clases de Animales" y "cada instancia de un Border Collie es una instancia de un Perro" serían, en este caso, mejores pruebas para subtipos.

Arriba

Asociaciones calificadas

Una **asociación calificada** es el equivalente en UML de un concepto de programación que se conoce de diferentes modos: arreglos asociativos, mapas y diccionarios.



Figura 5-10: *Asociación calificada*

Figura 5-10: Asociación calificada

La Figura 5-10 muestra un modo de representar la asociación entre las clases Pedido y Línea de pedido que emplea un calificador. El calificador dice que, en conexión con un Pedido, puede haber una Línea de pedido para cada instancia del Producto.

Conceptualmente, este ejemplo indica que no se puede tener dos instancias de Línea de pedido para el mismo Producto dentro de un Pedido. Desde la perspectiva de especificación, esta asociación calificada implicaría una interfaz semejante a

```
class Pedido {

   public Lineapedido articuloLinea (Producto unProducto)¡

   public void agregaArticuloLinea (Numero cantidad,

               Producto paraProducto);
```

Por tanto, todos los accesos a un Artículo de línea dado requieren como argumento la identidad de un Producto. Una multiplicidad de [1] indicaría que debe haber un Artículo de línea para cada Producto; [*] indicaría que se pueden tener varias Líneas de pedido por Producto, pero que el acceso a los Artículos de línea sigue estando indizado por Producto.

Desde una perspectiva de implementación, lo anterior sugiere el empleo de un arreglo asociativo o de una estructura de datos similar para contener las líneas del pedido.

```
Class Pedido {

    private Dictionary _articulosLinea;
```

En el modelado conceptual, yo utilizo el artificio calificador únicamente para mostrar restricciones del tipo "una sola Línea de pedido por Producto en el Pedido." En el caso de los modelos de especificación, me sirve para mostrar una interfaz de búsqueda indizada. Me siento cómodo manejando esto y asociación no calificada al mismo tiempo, si se trata de una interfaz adecuada.

Utilizo los calificadores dentro de los modelos de realización para mostrar los usos de un arreglo asociativo o de una estructura de datos similar.

Arriba

Clase de asociación

Las clases de asociación permiten añadir atributos, operaciones y otras características a las asociaciones, como se muestra en la Figura 5-11.

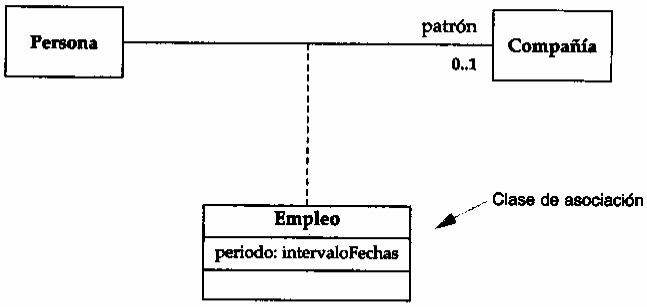


Figura 5-11: *Clase de asociación*

Figura 5-11:Clase de asociación

El diagrama nos permite apreciar que una Persona puede trabajar para una sola Compañía. Necesitamos conservar la información sobre el periodo de tiempo que trabaja cada empleado para cada Compañía.

Para lograrlo, añadimos un atributo de intervaloFechas a la asociación. Podríamos agregar este atributo a la clase Persona, pero, en realidad, es un hecho sobre la relación entre una Persona y una Compañía, la cual cambiará si también lo hiciera el patrón de la persona.

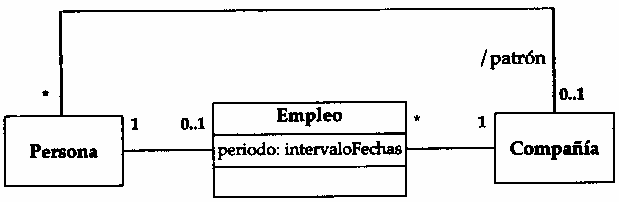


Figura 5-12: *Promoción de una clase de asociación a una clase completa*

Figura 5-12:Promoción de una clase de asociación a una clase completa

La Figura 5-12 muestra otra manera de representar esta información: convertirEmpleo en una clase completa por derecho propio (obsérvese cómo las multiplicidades se han movido en consecuencia). En este caso, cada una de las clases de la asociación original tiene una función de un solo valor con relación a la clase Empleo. El papel del "patrón" es ahora derivado, aunque no sea necesario mostrar esto.

¿Qué beneficio se logra con la clase de asociación que compense la notación extra que hay que recordar? La clase de asociación añade una restricción nueva, en el sentido de que sólo puede haber una instancia de la clase de asociación entre cualesquiera de dos objetos participantes. Me parece que hace falta un ejemplo.

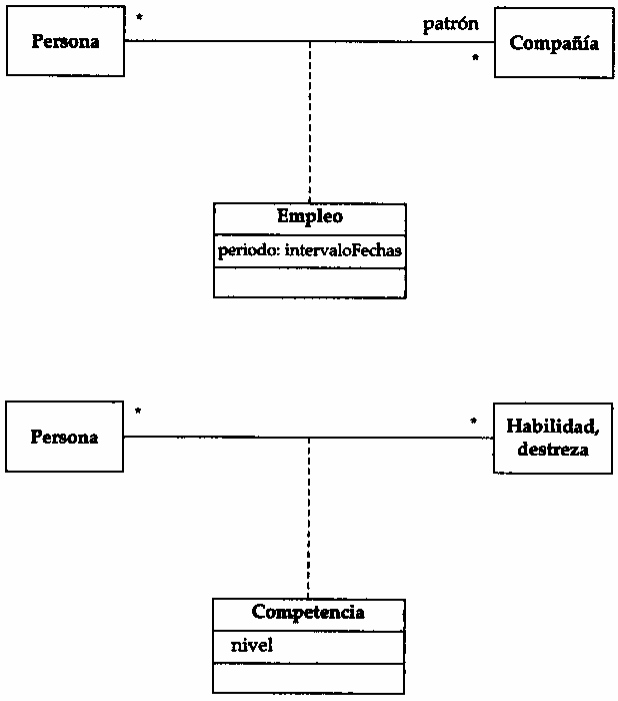


Figura 5-13: *Sutilezas de la clase de asociación*

Figura 5-13: Sutilezas de la clase de asociación

Observe por un momento los dos diagramas que aparecen en la Figura 5-13. Ambos tienen formas muy parecidas. Sin embargo, podríamos imaginamos a una Persona trabajando para la misma Compañía en diferentes periodos de tiempo (es decir, la persona deja la compañía y después vuelve). Esto significa que una Persona puede tener más de una asociación de Empleo con la misma Compañía a través del tiempo.

En lo que se refiere a las clases Persona y Oficio, sería difícil apreciar por qué una persona tendría más de una Competencia en el mismo oficio; de hecho, probablemente se consideraría como un error.

En el UML sólo es legal el segundo caso. Se puede tener solamente una Competencia para cada combinación de Persona y Oficio. El diagrama superior de la Figura 5-13 no permitiría a una Persona tener más de un Empleo en la misma Compañía. Si es necesario permitir esto, habrá que convertir a Empleo en una clase completa, a la manera de la Figura 5-12

En el pasado, los modeladores tenían varias premisas sobre el significado de una clase de asociación en estas circunstancias. Algunos suponían que sólo podían tenerse combinaciones únicas (como es el caso de la competencia); mientras que otros más no suponían tal restricción. Muchos no lo tomaban en consideración en lo absoluto y podrían asumir la restricción en algunos lugares y en otros no. Así pues, cuando utilice el UML, recuerde que la restricción siempre está allí.

Con frecuencia, se encuentra este tipo de artificio con información histórica, como en el caso de Empleo anterior. Aquí, un patrón útil es el de Mapa Histórico. Podemos usado definiendo un estereotipo «historia» (véase la figura 5-14).

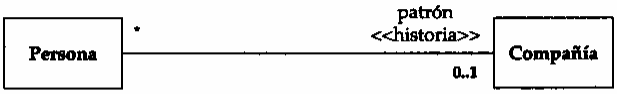


Figura 5-14: *Estereotipo de historia para las asociaciones*

Figura 5-14:Estereotipo de historia para las asociaciones

El modelo indica que una Persona únicamente puede trabajar para una sola Compañía en un momento dado. Sin embargo, a través de un tiempo determinado, una Persona puede trabajar para varias Compañías. Esto sugiere una interfaz semejante a las líneas siguientes

```
class Persona {

    // obtiene el patrón actual

    Compañía patrón();
```

```
    // patrón en una fecha dada

    Compañía patrón(Date);

    void cambiaPatron(Compañía nuevopatron, Date cambiaFecha);

    void dejaPatron (Date cambiaFecha):
```

El estereotipo «historia» no forma parte del UML, pero lo menciono aquí por dos razones. Primera, es un concepto que he encontrado útil en varias ocasiones durante mi ejercicio como modelador. Segundo, muestra cómo se pueden usar los estereotipos para ampliar el UML.

Arriba

Clase con parámetro

Algunos lenguajes, particularmente C ++, tienen el concepto de **clase con parámetro** (también conocida como **plantilla).**

La utilidad más obvia de este concepto es en el trabajo con colecciones en un lenguaje con especificación estricta de tipos de datos. De este modo, se puede definir la conducta de los conjuntos en general por medio de la definición de una clase de plantillas Conjunto.

```
class Conjunto <T> {

    void insert (T nuevoElemento) ;

    void remove (T unElemento);
```

Hecho esto, se podrá emplear la definición general para hacer clases de conjuntos para elementos más específicos.

```
Conjunto <Empleado> conjuntoEmpleados;
```

Una clase con parámetro en UML se declara mediante la notación que aparece en la Figura 5-15.

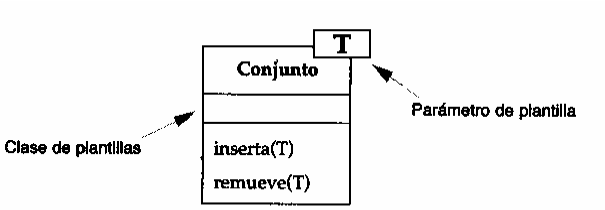


Figura 5-15: *Clase con parámetro*

Figura 5-15: Clase con parámetro

La T en el diagrama es un marcador para el parámetro de tipo (se podrá tener más de uno). En un lenguaje sin tipos, como Smalltalk, esta cuestión no surge, por lo que el concepto carece de utilidad.

Al uso de una clase con parámetro, tal como Conjunto<Empleado> mostrado antes, se le llama **elemento enlazado** (*bound*).

Se puede mostrar un elemento enlazado de dos maneras. La primera refleja la sintaxis C++ (véase la figura 5-16).

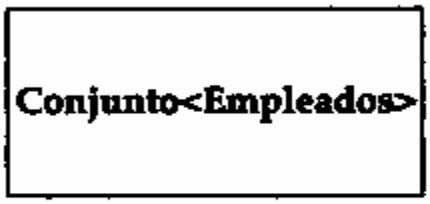


Figura 5-16: *Elemento enlazado (versión 1)*

Figura 5-16: Elemento enlazado (versión 1)

La anotación alternativa (véase la figura 5-17) retuerza el vínculo con la plantilla y permite renombrar el elemento enlazado.

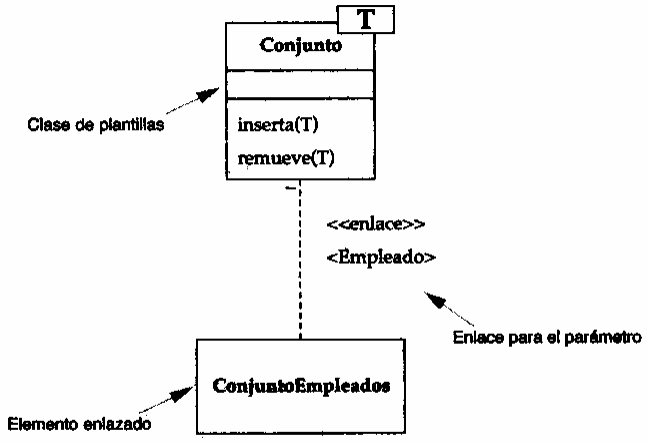


Figura 5-17: *Elemento enlazado (versión 2)*

Figura 5-17:Elemento enlazado (versión 2)

El estereotipo «vínculo» es un estereotipo en la relación de refinamiento. Esta relación indica que el ConjuntoEmpleados se conformará con la interfaz de Conjunto. En términos de especificación, el ConjuntoEmpleados es un subtipo de Conjunto. Esto se ajusta al otro modo de implementar las colecciones especificas de tipos, que consiste en declarar todos los subtipos apropiados.

Sin embargo, usar un elemento enlazado no es lo mismo que subtipificar. No se permite añadir características al elemento enlazado: es especificado en su totalidad por su plantilla; sólo se está añadiendo información de tipo restrictivo. Si se desea agregar características, habrá que crear un subtipo.

Aunque el empleo clásico de las clases con parámetros es con colecciones, hay muchas otras maneras de utilizarlas en C++

Las clases con parámetros le permiten a usted usar **tipificación derivada.** Cuando se escribe el cuerpo de la plantilla, es posible invocar operaciones sobre el parámetro. Posteriormente, cuando se declara un elemento enlazado, el compilador trata de asegurarse de que el parámetro que se ha proporcionado maneje las operaciones requeridas por la plantilla.

Éste es un mecanismo de tipificación derivada, pues no hay necesidad de definir un tipo para el parámetro; el compilador determina si el enlace es viable viendo a la fuente de la plantilla. Esta propiedad es medular para las clases con parámetros en el STL de C ++; estas clases pueden servir, además, para otros trucos interesantes.

El empleo de clases con parámetros tiene repercusiones. Por ejemplo, pueden causar una considerable inflación del código en C ++. Por mi parte, pocas veces incluyo clases con parámetros en el modelado conceptual, ante todo porque principalmente se usan para las colecciones, las cuales se dan a entender mediante asociaciones. Sólo uso clases con

parámetros en el modelado de especificación y de implementación, si son manejadas por el lenguaje con el que estoy trabajando.

Arriba

La visibilidad

Debo confesar que esta sección me produce cierta perturbación.

La **visibilidad** es uno de esos temas que, en principio, es simple, pero tiene sutilezas complejas. La idea básica es que cualquier clase tiene elementos públicos y privados. Los elementos públicos pueden ser usados por cualquier otra clase; los elementos privados sólo pueden ser usados por la clase propietaria. Sin embargo, cada lenguaje tiene sus propias reglas. Aunque todos los lenguajes utilizan términos como " **público** ", " **privado** " y " **protegido** " (*public, private, protected*), su significado varía según el lenguaje. Estas diferencias son pequeñas, pero generan confusión, en especial para quienes trabajamos con más de un lenguaje.

El UML trata de resolver este problema, sin complicarse demasiado. En esencia, en el UML se puede etiquetar cualquier atributo u operación con un indicador de visibilidad. Se puede usar el marcador que se quiera y su significado depende del lenguaje. Sin embargo, el UML proporciona tres abreviaciones de visibilidad (un tanto difíciles de recordar): + (público), - (privado), y # (protegido).

Siento la tentación de dejar aquí el tema pero, por desgracia, cada uno dibuja la visibilidad de una manera específica. Por tanto, para entender en realidad algunas de las diferencias comunes que existen entre los modelos es necesario comprender los enfoques que adoptan los diversos lenguajes, con respecto a la visibilidad. Así pues, aspiremos profundo y sumerjámonos en el fango.

Iniciamos con C++, ya que es la base del empleo regular del UML.

- Un miembro público es visible en todo el programa y puede ser llamado por cualquier objeto dentro del sistema.
- Un miembro privado sólo puede ser usado por la clase que lo define.
- Un miembro protegido sólo puede ser usado por (a) la clase que lo define, o (b) una subclase de esa clase.

Considérese una clase Cliente que tiene una subclase Cliente personal. Considérese también el objeto Martín, que es una instancia de Cliente personal. Martín puede usar cualquier miembro público de cualquier objeto del sistema. También puede usar cualquier miembro privado de la clase Cliente personal. Martín no puede usar ningún miembro privado

definido en Cliente; sin embargo, sí puede usar los miembros protegidos de Cliente y los usuarios protegidos de Cliente personal.

Java es similar a C++ en el hecho de que ofrece libre acceso a los miembros de otros objetos de su misma clase. Java también añade un nuevo nivel de visibilidad al que llama paquete (package). Un miembro con visibilidad de paquete sólo puede ser accedido por instancias de otras clases dentro del mismo paquete.

Siguiendo con nuestro tema, para asegurar que las cosas no sean tan sencillas, Java redefine levemente la visibilidad protegida. En Java, a un miembro protegido puede accederse por subclases, pero también por cualquier otra clase que esté en el mismo paquete que la clase propietaria. Esto quiere decir que, en Java, protegido es más público que paquete.

Java también permite que las clases sean marcadas como públicas o paquetes. Los miembros públicos de una clase pública pueden ser usados por cualquier clase que importe el paquete al que pertenezca la clase. Una clase de paquete puede ser usada sólo por otras clases del mismo paquete.

C ++ añade el toque final. Un método o clase C ++ puede ser convertido en amigo de una clase. Un **amigo** (*friend*) tiene completo acceso a todos los miembros de una clase. De aquí viene la frase: "en C++, los amigos se tocan entre ellos las partes privadas".

Cuando esté manejando visibilidad, emplee las reglas del lenguaje con que trabaja. Cuando vea un modelo UML que provenga de otra parte, desconfíe del significado de los marcadores de visibilidad y tenga presente la manera en que pueden cambiar tales significados de un lenguaje a otro.

En general, encuentro que las visibilidades cambian a medida que se trabaja en el código, así que no las tome demasiado en cuenta, desde un principio.

Arriba 

Características del alcance de clase

Se pueden definir operaciones o atributos en el nivel de alcance de clase.

Esto significa que éstos son característicos de la clase, en lugar de ser de cada objeto. Estas operaciones o atributos corresponden a los miembros estáticos en C ++ y Java y a los métodos y variables de clase en Smalltalk. Estas características se muestran de la misma manera que cualquier otra operación o atributo, a excepción de que se subrayan.

Arriba

Para mayor información

Arriba

Índice

- Agregación
- Amigo (Visibilidad)
- Atributos derivados
- Asociación calificada
- Asociaciones derivadas
- Clase abstracta
- clase con parámetro
- Clasificación
- clasificaciones
- Clasificación simple
- Clasificación múltiple
- clasificación dinámica
- clasificación estática
- composición
- Congelado
- elemento enlazado
- Estereotipos
- generalizaciones
- Herencia múltiple
- objetos inmutables
- objetos de referencia
- objetos de valor
- plantilla
- privado (Visibilidad)
- protegido (Visibilidad)
- pública (Visibilidad)
- refinamiento
- relación de valor múltiple
- tipificación derivada
- visibilidad

Arriba 